JAVA OOPs

---

1. Object-Oriented Programming (OOP) Concepts in Java

Definition / Introduction

Object-Oriented Programming (OOP) is a programming approach where programs are organized using objects and classes. Java is fully object-oriented (mostly).

Key Concepts / Terms

- **Class**
- **Object**
- **Encapsulation**
- **Abstraction**
- **Inheritance**
- **Polymorphism**

Explanation / Working Principle

In OOP, a program is designed using real-world entities called objects. Each object contains:

- **Data (variables)**
- **Behavior (methods)**

Objects interact with each other to complete tasks.

Types / Classification

- **Class-based OOP (Java, C++)**
- **Prototype-based (JavaScript)**

Advantages

- **Easy to understand**

- **Code reusability**
- **Better security**
- **Modular design**

Disadvantages

- **Slightly complex for beginners**
- **More memory usage**

Applications

- **Banking systems**
- **Mobile applications**
- **Games**
- **Web applications**

Conclusion / Summary

OOP makes programming more structured, reusable, and closer to real-world modeling.

---

2. Compare Procedural vs Object-Oriented Programming

Definition / Introduction

Procedural programming focuses on functions, while OOP focuses on objects.

Key Differences

Feature	Procedural	OOP
Approach	Top-down	Bottom-up
Focus	Functions	Objects
Data Security	Low	High
Reusability	Limited	High
Example Languages	C	Java, C++

Explanation / Working Principle

Procedural programming solves problems step-by-step using functions. OOP solves problems using interacting objects.

Advantages of OOP

- **Better data security**
- **Code reusability**
- **Easy maintenance**
- **Real-world modeling**

Conclusion / Summary

OOP is more powerful and widely used than procedural programming.

---

3. Classes and Objects with Example

Definition / Introduction

- **Class: Blueprint of an object**
- **Object: Instance of a class**

Key Concepts

- **Class defines structure**
- **Object represents real entity**

Explanation / Working Principle

A class defines variables and methods. Objects are created from classes to use them.

Example

```
class Car {
 String color;

 void drive() {
```

```
    System.out.println("Car is driving");
  }
}

public class Main {
  public static void main(String[] args) {
    Car c1 = new Car();
    c1.color = "Red";
    c1.drive();
  }
}
```

Advantages

- **Code reusability**
- **Easy modeling of real-world systems**

Applications

- **Software development**
- **Simulation systems**

Conclusion / Summary

Class defines structure, and object is the real working entity.

---

4. Encapsulation, Abstraction, and Inheritance with Examples

Definition / Introduction

These are the three main pillars of OOP.

---

1. Encapsulation

Definition

Wrapping data and methods into a single unit.

Example

```
class Student {
 private int age;

 public void setAge(int a) {
  age = a;
 }
}
```

Real-life Example

Medicine capsule (data hidden inside)

---

2. Abstraction

Definition

Hiding internal implementation and showing only functionality.

Example

ATM machine (user does not see internal processing)

---

3. Inheritance

Definition

One class acquires properties of another class.

Example

```
class Animal {
 void eat() {
  System.out.println("Eating");
 }
}

class Dog extends Animal {
}
```

Real-life Example

Child inherits properties from parents

Conclusion / Summary

Encapsulation protects data, abstraction hides details, and inheritance allows reuse of code.

---

5. Benefits of Inheritance in Java

Definition / Introduction

Inheritance is a feature where one class acquires properties and behaviors of another class.

Key Concepts

- **Parent class (Super class)**
- **Child class (Sub class)**

Explanation / Working Principle

A child class can reuse methods and variables of a parent class using the "extends" keyword.

Advantages / Merits

- **Code reusability**
- **Reduces repetition**
- **Easier maintenance**
- **Supports hierarchical classification**
- **Improves readability**

Disadvantages / Limitations

- **Tight coupling between classes**
- **Improper use may create confusion**

Applications

- **Software development frameworks**
- **Banking systems hierarchy**
- **Game development (characters)**

Conclusion / Summary

Inheritance improves code reuse and reduces redundancy in Java programs.

---

6. Polymorphism and Types of Polymorphism

Definition / Introduction

Polymorphism means “many forms”. It allows one action to behave differently in different situations.

Key Concepts

- **Same method name**
- **Different behavior**

Explanation / Working Principle

A single function or method can perform multiple tasks depending on input or object.

Types / Classification

1. **Compile-time Polymorphism (Method Overloading)**
2. **Runtime Polymorphism (Method Overriding)**

Formula / Rules

- **Overloading: Same method name, different parameters**
- **Overriding: Same method signature in parent and child class**

Advantages

- **Increases flexibility**
- **Improves code readability**
- **Supports dynamic behavior**

Disadvantages

- **Can be confusing for beginners**

Applications

- **Banking systems**
- **GUI systems**
- **Game development**

Conclusion / Summary

Polymorphism makes Java flexible and dynamic.

---

7. Java History and Features of Java

Definition / Introduction

Java is a high-level programming language developed by Sun Microsystems in 1995.

Key Concepts

- **Developed by James Gosling**
- **Originally called Oak**

Explanation / Working Principle

Java works on Write Once, Run Anywhere (WORA) principle using JVM (Java Virtual Machine).

Features of Java

- **Simple**
- **Object-oriented**

- **Platform independent**
- **Secure**
- **Robust**
- **Portable**
- **Multithreaded**

Advantages

- **Cross-platform support**
- **High security**
- **Easy to learn**

Disadvantages

- **Slower than C++**
- **Requires JVM**

Applications

- **Web applications**
- **Mobile apps (Android)**
- **Enterprise software**

Conclusion / Summary

Java is a powerful, platform-independent programming language.

---

8. Variables, Constants, and Data Types in Java

Definition / Introduction

Variables store data, constants store fixed values, and data types define the type of data.

Key Concepts

- **Variable: Changeable value**

- **Constant: Fixed value (final keyword)**
- **Data type: Type of data stored**

Explanation / Working Principle

Java assigns memory based on data types.

Types / Classification

1. Variables

- **Local variable**
- **Instance variable**
- **Static variable**

2. Data Types

- **Primitive: int, float, char, boolean**
- **Non-primitive: String, Arrays, Classes**

Formula / Syntax

```
final int x = 10; // constant
int a = 5;        // variable
```

Advantages

- **Better memory management**
- **Code clarity**

Disadvantages

- **Wrong type causes errors**

Applications

- **All Java programs**

Conclusion / Summary

Variables store changing data, constants store fixed data, and data types define data behavior.

---

9. Scope and Lifetime of Variables

Definition / Introduction

Scope means the area where a variable can be accessed. Lifetime means how long a variable exists in memory.

Key Concepts

- **Scope: Visibility of variable**
- **Lifetime: Duration of variable in memory**

Explanation / Working Principle

Variables are created when declared and destroyed when they go out of scope.

Types / Classification

- **Local variables (inside method)**
- **Instance variables (inside class)**
- **Static variables (shared across class)**

Advantages

- **Better memory usage**
- **Avoids variable conflicts**

Disadvantages

- **Improper scope may cause errors**

Applications

- **All Java programs**

Conclusion / Summary

Scope controls visibility, and lifetime controls memory duration.

---

10. Operators and Operator Hierarchy in Java

Definition / Introduction

Operators are symbols used to perform operations on variables and values.

Key Concepts

- **Operand: Value or variable**
- **Operator: Symbol (+, -, *, etc.)**

Explanation / Working Principle

Operators work on operands to perform calculations or logical operations.

Types / Classification

- **Arithmetic (+, -, *, /)**
- **Relational (==, !=, >, <)**
- **Logical (&&, ||, !)**
- **Assignment (=, +=, -=)**

Operator Hierarchy (Precedence)

1. **()**
2. ***, /, %**
3. **+, -**
4. **Relational**
5. **Logical**
6. **Assignment**

Advantages

- **Easy calculations**
- **Fast operations**

Disadvantages

- **Wrong precedence causes errors**

Applications

- **All programming logic**

Conclusion / Summary

Operators perform actions, and hierarchy defines execution order.

---

11. Type Conversion and Casting in Java

Definition / Introduction

Type conversion is changing one data type into another.

Key Concepts

- **Widening (automatic)**
- **Narrowing (manual)**

Explanation / Working Principle

Java automatically converts smaller types to larger types or requires explicit casting.

Types / Classification

1. **Implicit conversion (automatic)**
2. **Explicit casting (manual)**

Formula / Syntax

```
int a = 10;
double b = a; // implicit

int x = (int) 10.5; // explicit
```

Advantages

- **Flexible data handling**

Disadvantages

- **Data loss in narrowing conversion**

Applications

- **Mathematical operations**

Conclusion / Summary

Type conversion allows compatibility between different data types.

---

12. Enumerated Types in Java (Enum)

Definition / Introduction

Enum is a special data type that represents a group of constants.

Key Concepts

- **Fixed set of values**
- **Type-safe constants**

Explanation / Working Principle

Enum defines predefined constant values that cannot change.

Types / Classification

- **Simple enum**
- **Enum with methods**

Formula / Syntax

```
enum Day {
 MONDAY, TUESDAY, WEDNESDAY
}
```

Advantages

- **Improves code readability**
- **Type safety**
- **Easy maintenance**

Disadvantages

- **Less flexible than variables**

Applications

- **Days of week**
- **Directions**
- **Status codes**

Conclusion / Summary

Enum is used to define fixed constant values in Java.

---

13. Control Flow Statements in Java

Definition / Introduction

Control flow statements are used to control the execution order of program statements.

Key Concepts

- **Decision making**
- **Looping**
- **Branching**

Explanation / Working Principle

They allow the program to take decisions and repeat actions based on conditions.

Types / Classification

1. Selection Statements

- **if**
- **if-else**
- **switch**

2. Looping Statements

- **for**
- **while**
- **do-while**

3. Jump Statements

- **break**
- **continue**
- **return**

Advantages

- **Controls program flow**
- **Reduces complexity**

Disadvantages

- **Improper use may create infinite loops**

Applications

- **All Java programs**

Conclusion / Summary

Control statements help in decision making and repeating tasks.

---

14. Break and Continue Statements in Java

Definition / Introduction

break and continue are jump statements used to control loops.

Key Concepts

- **break: exits loop**
- **continue: skips current iteration**

Explanation / Working Principle

- **break stops loop completely**
- **continue skips current step and moves to next iteration**

Types / Classification

- **break statement**
- **continue statement**

Formula / Syntax

```
break;
continue;
```

Advantages

- **Improves control in loops**
- **Helps avoid unnecessary execution**

Disadvantages

- **Overuse makes code confusing**

Applications

- **Searching algorithms**
- **Loop control**

Conclusion / Summary

break stops loop, continue skips iteration.

15. Arrays in Java

Definition / Introduction

An array is a collection of similar type elements stored in continuous memory.

Key Concepts

- **Index-based storage**
- **Fixed size**

Explanation / Working Principle

Arrays store multiple values under one variable name using indexes starting from 0.

Types / Classification

- **One-dimensional array**
- **Two-dimensional array**

Formula / Syntax

```
int arr[] = new int[5];
```

Advantages

- **Easy access using index**
- **Efficient storage**

Disadvantages

- **Fixed size**
- **Insertion and deletion is difficult**

Applications

- **Data storage**
- **Sorting algorithms**

Conclusion / Summary

Arrays store multiple values in a single variable.

---

16. Console Input and Output in Java

Definition / Introduction

Console I/O is used to take input from user and display output.

Key Concepts

- **Input: Scanner class**
- **Output: System.out**

Explanation / Working Principle

Java uses Scanner class for input and System.out for output.

Types / Classification

- **Input methods**
- **Output methods**

Formula / Syntax

```
Scanner sc = new Scanner(System.in);
int a = sc.nextInt();
System.out.println(a);
```

Advantages

- **Easy user interaction**

Disadvantages

- **Slower than GUI input**

Applications

- **Basic programs**
- **User input systems**

Conclusion / Summary

Console I/O is used for simple user interaction in Java.

---

17. Formatted Output in Java

Definition / Introduction

Formatted output means displaying data in a structured and readable format.

Key Concepts

- **Formatting numbers**
- **Controlling decimal places**
- **Aligning output**

Explanation / Working Principle

Java uses printf() method to format output using format specifiers.

Types / Classification

- **Integer formatting**
- **Float formatting**
- **String formatting**

Formula / Syntax

```
System.out.printf("Value = %.2f", 10.567);
```

Advantages

- **Clean output**
- **Better readability**

Disadvantages

- **Slightly complex syntax**

Applications

- **Reports**
- **Billing systems**

Conclusion / Summary

Formatted output improves readability of data display.

---

18. Constructors and Methods in Java

Definition / Introduction

Constructor is used to initialize objects, and methods define behavior.

Key Concepts

- **Constructor name same as class**
- **Method performs actions**

Explanation / Working Principle

Constructor runs automatically when object is created, while methods are called explicitly.

Types / Classification

Constructors

- **Default constructor**
- **Parameterized constructor**

Methods

- **User-defined methods**
- **Predefined methods**

Formula / Syntax

```
class A {
  A() { } // constructor
```

```
    void show() { } // method
}
```

Advantages

- **Easy object initialization**
- **Code reuse**

Disadvantages

- **Constructors cannot return values**

Applications

- **Object creation**
- **Data initialization**

Conclusion / Summary

Constructors initialize objects and methods define behavior.

---

19. Garbage Collection in Java

Definition / Introduction

Garbage collection is the process of automatically removing unused objects from memory.

Key Concepts

- **Automatic memory management**
- **JVM responsibility**

Explanation / Working Principle

When objects are no longer referenced, JVM removes them to free memory.

Types / Classification

- **Minor GC**
- **Major GC**

Advantages

- **No manual memory management**
- **Prevents memory leaks**

Disadvantages

- **No control over timing**
- **May affect performance**

Applications

- **All Java applications**

Conclusion / Summary

Garbage collection automatically manages memory in Java.

---

20. Inheritance in Java (Types and Hierarchy)

Definition / Introduction

Inheritance is a mechanism where one class acquires properties of another class.

Key Concepts

- **Super class (parent)**
- **Sub class (child)**
- **“extends” keyword**

Explanation / Working Principle

A child class inherits variables and methods of parent class to reuse code.

Types / Classification

- **Single inheritance**
- **Multilevel inheritance**

- **Hierarchical inheritance**
- **Multiple inheritance (via interfaces)**

Formula / Syntax

```
class A { }
class B extends A { }
```

Advantages

- **Code reusability**
- **Reduces duplication**
- **Easy maintenance**

Disadvantages

- **Tight coupling**
- **Complexity increases in deep hierarchy**

Applications

- **Banking systems**
- **Game character design**

Conclusion / Summary

Inheritance helps in reusing and organizing code efficiently in Java.

---

21. Polymorphism with Method Overriding and Dynamic Binding

Definition / Introduction

Polymorphism means “many forms”. It allows the same method to behave differently based on the object.

Key Concepts

- **Method Overriding**
- **Dynamic binding**

- **Runtime behavior**

Explanation / Working Principle

In runtime polymorphism, a parent reference refers to a child object and method execution is decided at runtime.

Types / Classification

- **Compile-time polymorphism (method overloading)**
- **Runtime polymorphism (method overriding)**

Formula / Syntax

```
class A {
 void show() { }
}
class B extends A {
 void show() { }
}
```

Advantages

- **Flexibility**
- **Code reusability**
- **Dynamic behavior**

Disadvantages

- **Slight complexity**

Applications

- **Banking systems**
- **Game development**

Conclusion / Summary

Polymorphism allows one interface to have multiple behaviors.

---

22. Abstract Classes vs Interfaces

Definition / Introduction

Abstract class and interface are used to achieve abstraction in Java.

Key Concepts

- **Abstract class: partial abstraction**
- **Interface: full abstraction**

Explanation / Working Principle

Abstract classes can have both abstract and concrete methods, while interfaces only define method signatures (before Java 8).

Differences

Feature	Abstract Class	Interface
Methods	Abstract + concrete	Only abstract (mostly)
Variables	Allowed	Only constants
Inheritance	Single	Multiple

Advantages

- **Achieves abstraction**
- **Improves design**

Disadvantages

- **Interface can be rigid**

Applications

- **Designing frameworks**
- **API design**

Conclusion / Summary

Abstract class gives partial abstraction, interface gives full abstraction.

23. Interfaces and Implementation in Java

Definition / Introduction

An interface is a blueprint of a class that contains abstract methods.

Key Concepts

- **Implements keyword**
- **Multiple inheritance support**

Explanation / Working Principle

A class implements an interface and provides method definitions.

Types / Classification

- **Simple interface**
- **Multiple interfaces implementation**

Formula / Syntax

```
interface A {
  void show();
}
class B implements A {
  public void show() { }
}
```

Advantages

- **Supports multiple inheritance**
- **Loose coupling**

Disadvantages

- **Requires full implementation**

Applications

- **Design systems**
- **Event handling**

Conclusion / Summary

Interfaces define rules that classes must follow.

---

24. Packages in Java (Creation, Use, CLASSPATH)

Definition / Introduction

A package is a group of related classes and interfaces.

Key Concepts

- **Built-in packages**
- **User-defined packages**
- **CLASSPATH**

Explanation / Working Principle

Packages help organize code and avoid naming conflicts.

Types / Classification

- **Built-in packages (java.util, java.io)**
- **User-defined packages**

Formula / Syntax

```
package mypack;
class A { }
```

Advantages

- **Code organization**
- **Reusability**
- **Avoids naming conflicts**

Disadvantages

- **Complex setup for beginners**

Applications

- **Large software systems**

Conclusion / Summary

Packages organize Java classes in a structured way.

---

25. super Keyword in Java

Definition / Introduction

The super keyword is used to refer to the immediate parent class object.

Key Concepts

- **Parent class reference**
- **Used in inheritance**

Explanation / Working Principle

It is used to access parent class variables, methods, and constructors.

Types / Uses

- **Access parent variable: super.var**
- **Call parent method: super.method()**
- **Call parent constructor: super()**

Formula / Syntax

```
class A {
 int x = 10;
}
class B extends A {
 void show() {
  System.out.println(super.x);
```

```
    }
}
```

Advantages

- **Avoids ambiguity**
- **Access parent class features**

Disadvantages

- **Can only refer to immediate parent**

Applications

- **Method overriding cases**

Conclusion / Summary

super keyword is used to access parent class members.

---

26. final Class and final Method

Definition / Introduction

final keyword is used to restrict inheritance, method overriding, or value modification.

Key Concepts

- **final class: cannot be inherited**
- **final method: cannot be overridden**

Explanation / Working Principle

Once declared final, Java prevents modification or extension.

Types / Classification

- **final variable**
- **final method**
- **final class**

Formula / Syntax

```
final class A { }
class B extends A { } // error
```

Advantages

- Security
- Prevents modification

Disadvantages

- Reduces flexibility

Applications

- Security classes
- Constants usage

Conclusion / Summary

final keyword prevents modification of classes and methods.

---

27. Inner Class and Its Types

Definition / Introduction

An inner class is a class defined inside another class.

Key Concepts

- Nested class
- Outer class

Explanation / Working Principle

Inner class can access members of outer class.

Types / Classification

- Member inner class
- Static nested class
- Local inner class

- **Anonymous inner class**

Formula / Syntax

```
class Outer {
  class Inner { }
}
```

Advantages

- **Better encapsulation**
- **Logical grouping**

Disadvantages

- **Complex structure**

Applications

- **Event handling**
- **GUI programming**

Conclusion / Summary

Inner classes improve organization and encapsulation.

---

28. Object Class in Java

Definition / Introduction

Object class is the root class of all Java classes.

Key Concepts

- **All classes inherit Object class**
- **Default parent class**

Explanation / Working Principle

Every class in Java directly or indirectly inherits Object class methods.

Important Methods

- **toString()**
- **equals()**
- **hashCode()**
- **getClass()**

Advantages

- **Common behavior for all objects**

Disadvantages

- **Sometimes overridden for customization**

Applications

- **Core Java architecture**

Conclusion / Summary

Object class is the parent of all Java classes.

---

29. Difference Between Abstract Class and Interface

Definition / Introduction

Abstract class and interface are both used to achieve abstraction in Java.

Key Concepts

- **Abstract class: partial abstraction**
- **Interface: full abstraction (traditional)**

Differences

Feature	Abstract Class	Interface
Methods	Abstract + concrete	Only abstract methods (mostly)

Feature	Abstract Class	Interface
Variables	Any type	Only constants (public static final)
Inheritance	Single inheritance	Multiple inheritance supported
Constructor	Allowed	Not allowed

Advantages

- Helps in designing flexible systems

Disadvantages

- Interface can be rigid

Conclusion / Summary

Abstract class is partially abstract, interface is fully abstract blueprint.

---

30. Exception Handling Mechanism in Java

Definition / Introduction

Exception handling is a mechanism to handle runtime errors and maintain normal program flow.

Key Concepts

- Exception
- Try-catch block
- JVM handling

Explanation / Working Principle

When an error occurs, Java creates an exception object and transfers control to exception handler.

Types / Classification

- **Checked exceptions**
- **Unchecked exceptions**

Advantages

- **Prevents program crash**
- **Improves reliability**

Disadvantages

- **Adds complexity**

Applications

- **File handling**
- **Database operations**

Conclusion / Summary

Exception handling ensures smooth execution even in error conditions.

---

31. try, catch, finally, throw, throws in Java

Definition / Introduction

These are keywords used for handling exceptions in Java.

Key Concepts

- **try: block where error may occur**
- **catch: handles exception**
- **finally: always executes**
- **throw: manually throws exception**
- **throws: declares exception**

Explanation / Working Principle

Java tries code, catches errors, executes cleanup, and allows custom exception control.

Syntax

```
try {
  // risky code
} catch(Exception e) {
  // handling
} finally {
  // always executes
}
```

Advantages

- **Better error control**
- **Cleaner code flow**

Disadvantages

- **Overuse reduces readability**

Applications

- **File handling**
- **Network programming**

Conclusion / Summary

These keywords provide complete control over exception handling.

---

32. Exception Hierarchy (Checked vs Unchecked)

Definition / Introduction

Java exceptions are categorized into checked and unchecked exceptions.

Key Concepts

- **Throwable class**
- **Exception hierarchy**

Explanation / Working Principle

All exceptions inherit from Throwable class.

Types / Classification

1. Checked Exceptions

- **Checked at compile time**
- **Example: IOException, SQLException**

2. Unchecked Exceptions

- **Occur at runtime**
- **Example: NullPointerException, ArithmeticException**

Advantages

- **Structured error handling**

Disadvantages

- **Checked exceptions increase coding effort**

Applications

- **File systems**
- **Database systems**

Conclusion / Summary

Checked exceptions are compile-time, unchecked occur at runtime.

---

33. Multithreading and Thread Life Cycle in Java

Definition / Introduction

Multithreading is a process of executing multiple threads simultaneously in a program.

Key Concepts

- **Thread**
- **Lightweight process**
- **Concurrent execution**

Explanation / Working Principle

Java allows multiple threads to run at the same time to improve CPU utilization.

Thread Life Cycle

1. **New**
2. **Runnable**
3. **Running**
4. **Blocked/Waiting**
5. **Terminated**

Advantages

- **Better performance**
- **Efficient CPU usage**
- **Fast execution**

Disadvantages

- **Complex programming**
- **Risk of deadlock**

Applications

- **Games**
- **Web servers**

- **Real-time systems**

Conclusion / Summary

Multithreading improves performance by executing multiple tasks simultaneously.

---

34. Synchronization and Inter-Thread Communication (Producer-Consumer)

Definition / Introduction

Synchronization is a mechanism to control access of multiple threads to shared resources.

Key Concepts

- **Thread safety**
- **Monitor lock**
- **wait(), notify(), notifyAll()**

Explanation / Working Principle

Only one thread can access synchronized block at a time to avoid data inconsistency.

Producer-Consumer Problem

Producer produces data and consumer consumes it using shared buffer.

Advantages

- **Prevents data inconsistency**
- **Safe thread communication**

Disadvantages

- **Reduces performance**
- **Can cause deadlock**

Applications

- **Banking systems**
- **Resource sharing systems**

Conclusion / Summary

Synchronization ensures safe communication between threads.

---

35. Built-in Exceptions in Java

Definition / Introduction

Built-in exceptions are predefined exceptions provided by Java.

Key Concepts

- **Predefined error types**
- **Runtime and compile-time exceptions**

Common Examples

- **ArithmeticException**
- **NullPointerException**
- **ArrayIndexOutOfBoundsException**
- **IOException**

Explanation / Working Principle

Java automatically throws these exceptions when an error occurs.

Advantages

- **Easy debugging**
- **Predefined error handling**

Disadvantages

- **Limited customization**

Applications

- **All Java programs**

Conclusion / Summary

Built-in exceptions help handle common runtime errors.

---

36. User-Defined Exceptions in Java

Definition / Introduction

User-defined exceptions are custom exceptions created by programmers.

Key Concepts

- **extends Exception class**
- **Custom error handling**

Explanation / Working Principle

Developers define their own exception class to handle specific errors.

Formula / Syntax

```
class MyException extends Exception {
}
```

Advantages

- **Custom error messages**
- **Better control**

Disadvantages

- **Extra coding required**

Applications

- **Banking systems**
- **Validation systems**

Conclusion / Summary

User-defined exceptions allow customized error handling.

---

37. Difference Between Process and Thread

Definition / Introduction

A process is a program in execution, while a thread is a lightweight unit of a process.

Key Concepts

- **Process: Independent execution unit**
- **Thread: Sub-unit of process**

Differences

Feature	Process	Thread
Memory	Separate	Shared
Speed	Slower	Faster
Communication	Complex	Easy

Advantages

- **Threads improve efficiency**

Disadvantages

- **Threads may cause synchronization issues**

Conclusion / Summary

Process is heavy, thread is lightweight.

38. Thread Priority in Java

Definition / Introduction

Thread priority determines the execution order of threads.

Key Concepts

- **Priority range: 1 to 10**
- **Default priority: 5**

Explanation / Working Principle

Higher priority threads get more CPU time.

Types / Classification

- **MIN_PRIORITY (1)**
- **NORM_PRIORITY (5)**
- **MAX_PRIORITY (10)**

Syntax

```
thread.setPriority(10);
```

Advantages

- **Controls execution order**

Disadvantages

- **Not guaranteed by JVM**

Conclusion / Summary

Thread priority influences scheduling but not strictly enforced.

39. Interrupting a Thread in Java

Definition / Introduction

Interrupting a thread is used to stop or pause a running thread.

Key Concepts

- **interrupt() method**
- **Exception handling**

Explanation / Working Principle

A thread is interrupted using interrupt() method which signals it to stop execution.

Syntax

```
thread.interrupt();
```

Advantages

- **Controlled stopping of threads**

Disadvantages

- **Requires proper handling**

Conclusion / Summary

Interrupt method is used to stop threads safely.

---

40. Java Collection Framework (JCF) and Architecture

Definition / Introduction

Java Collection Framework is a set of classes and interfaces to store and manipulate data.

Key Concepts

- **Collection interface**
- **List, Set, Queue**
- **Map interface**

Explanation / Working Principle

JCF provides ready-made data structures for storing groups of objects.

Architecture

- **Collection Interface**
  - **List**
  - **Set**
  - **Queue**
- **Map (separate hierarchy)**

Advantages

- **Easy data handling**
- **Reusable structures**
- **Efficient algorithms**

Disadvantages

- **Memory overhead**

Applications

- **Data storage**
- **Sorting and searching**

Conclusion / Summary

JCF simplifies handling of group of objects in Java.

---

41. ArrayList, Vector, Stack, Hashtable Comparison

Definition / Introduction

These are collection classes used to store groups of objects in Java.

Key Concepts

- **ArrayList: dynamic array**
- **Vector: synchronized dynamic array**
- **Stack: LIFO structure**
- **Hashtable: key-value pair storage**

Comparison

Feature	ArrayList	Vector	Stack	Hashtable
Synchronization	No	Yes	Yes	Yes
Performance	Fast	Slow	Medium	Medium
Structure	List	List	Stack (LIFO)	Key-value

Advantages

- **Easy data storage**

Disadvantages

- **Some are slower due to synchronization**

Conclusion / Summary

These collections help store and manage data efficiently.

---

42. Iterator and Enumeration in Java

Definition / Introduction

Iterator and Enumeration are used to traverse collections.

Key Concepts

- **Iterator: modern cursor**
- **Enumeration: legacy cursor**

Differences

Feature	Iterator	Enumeration
Removal	Allowed	Not allowed
Speed	Faster	Slower

Explanation / Working Principle

Both are used to access elements one by one.

Advantages

- **Easy traversal**

Disadvantages

- **Enumeration is outdated**

Conclusion / Summary

Iterator is preferred over Enumeration.

---

43. File Handling using Byte and Character Streams

Definition / Introduction

File handling is used to read/write data in files using streams.

Key Concepts

- **Byte stream: binary data**
- **Character stream: text data**

Explanation / Working Principle

Byte streams handle raw data, character streams handle characters.

Types / Classification

Byte Stream

- **InputStream**
- **OutputStream**

Character Stream

- **Reader**
- **Writer**

Advantages

- **Efficient file handling**

Disadvantages

- **Complex for beginners**

Applications

- **File reading/writing**

Conclusion / Summary

Streams are used for file input and output in Java.

---

44. JDBC Architecture and Types of Drivers

Definition / Introduction

JDBC (Java Database Connectivity) is used to connect Java applications with databases.

Key Concepts

- **Driver**
- **Connection**
- **Statement**
- **ResultSet**

JDBC Architecture

Java Application → JDBC API → Driver Manager → Database

Types of Drivers

1. **Type 1: JDBC-ODBC bridge**
2. **Type 2: Native API**
3. **Type 3: Network Protocol**
4. **Type 4: Thin Driver (most used)**

Advantages

- **Database connectivity**
- **Platform independent**

Disadvantages

- **Complex setup**

Applications

- **Banking systems**
- **Web applications**

Conclusion / Summary

JDBC enables Java programs to interact with databases.

---

45. Steps to Connect Java Program with Database using JDBC

Definition / Introduction

JDBC allows Java programs to connect and interact with databases.

Key Concepts

- **Driver loading**
- **Connection**
- **Statement execution**

Steps / Procedure

1. **Load JDBC driver**
2. **Establish connection**
3. **Create statement**
4. **Execute query**
5. **Process results**
6. **Close connection**

Syntax

```
Connection con = DriverManager.getConnection(url, user, pass);
```

Advantages

- **Easy database access**

Disadvantages

- **Requires proper setup**

Conclusion / Summary

JDBC connection allows Java to interact with databases step-by-step.

---

46. Scanner Class in Java

Definition / Introduction

Scanner class is used to take input from the user.

Key Concepts

- **Found in java.util package**
- **Used for console input**

Explanation / Working Principle

It reads input from keyboard and converts it into required type.

Syntax

```
Scanner sc = new Scanner(System.in);
int x = sc.nextInt();
```

Advantages

- **Easy input handling**

Disadvantages

- **Slower than BufferedReader**

Applications

- **User input programs**

Conclusion / Summary

Scanner class is used for easy input in Java.

---

47. StringTokenizer in Java

Definition / Introduction

StringTokenizer is used to break a string into tokens.

Key Concepts

- **Token: small part of string**
- **Delimiter: separator**

Explanation / Working Principle

It splits string based on delimiters like space or comma.

Syntax

```
StringTokenizer st = new StringTokenizer("Hello World");
```

Advantages

- **Simple string splitting**

Disadvantages

- **Old class (less used now)**

Applications

- **Parsing strings**

Conclusion / Summary

StringTokenizer splits strings into smaller parts.

---

48. Difference Between Byte Stream and Character Stream

Definition / Introduction

Byte stream handles binary data, character stream handles text data.

Key Concepts

- **Byte stream: 8-bit data**
- **Character stream: 16-bit Unicode data**

Differences

Feature	Byte Stream	Character Stream
Data Type	Binary	Text
Classes	InputStream, OutputStream	Reader, Writer

Advantages

- Suitable for different data types

Disadvantages

- Misuse can cause data errors

Conclusion / Summary

Byte stream is for binary, character stream is for text data.

---

49. Random Class in Java

Definition / Introduction

Random class is used to generate random numbers in Java.

Key Concepts

- Found in java.util package
- Used for random values

Explanation / Working Principle

It generates pseudo-random numbers using built-in algorithms.

Syntax

```
Random r = new Random();
int x = r.nextInt(100);
```

Advantages

- Easy random number generation

Disadvantages

- **Not truly random (pseudo-random)**

Applications

- **Games**
- **Simulations**

Conclusion / Summary

Random class is used to generate random values in Java.

---

50. AWT vs Swing with Hierarchy

Definition / Introduction

AWT and Swing are used to create graphical user interfaces in Java.

Key Concepts

- **AWT: Abstract Window Toolkit (older)**
- **Swing: Advanced GUI toolkit**

Differences

Feature	AWT	Swing
Components	Heavyweight	Lightweight
Speed	Slower	Faster
Look	Platform dependent	Platform independent

Hierarchy

Component → Container → Window → Frame

Advantages

- **Swing is more flexible**

Disadvantages

- **AWT is outdated**

Conclusion / Summary

Swing is more powerful than AWT for GUI development.

---

51. Swing Components and Container Classes

Definition / Introduction

Swing provides advanced GUI components for Java applications.

Key Concepts

- **Lightweight components**
- **Containers**

Components

- **JButton**
- **JLabel**
- **JTextField**
- **JCheckBox**

Containers

- **JFrame**
- **JPanel**
- **JDialog**

Advantages

- **Rich GUI design**

Disadvantages

- **Slightly complex**

Applications

- **Desktop applications**

Conclusion / Summary

Swing is used to create modern GUI applications in Java.

---

52. Event Handling in Java (Delegation Model)

Definition / Introduction

Event handling is a mechanism to handle user actions like clicks and key presses.

Key Concepts

- **Event source**
- **Event listener**
- **Event object**

Explanation / Working Principle

User action generates event which is handled by listener.

Types / Classification

- **ActionEvent**
- **MouseEvent**
- **KeyEvent**

Syntax

```
button.addActionListener(e -> System.out.println("Clicked"));
```

Advantages

- **Interactive programs**

Disadvantages

- **Complex for beginners**

Applications

- **GUI applications**

Conclusion / Summary

Event handling connects user actions with program response.

---

53. Layout Managers in Java (Flow, Border, Grid)

Definition / Introduction

Layout managers control the arrangement of components in a GUI.

Key Concepts

- **Controls size and position**
- **Automatic layout handling**

Types / Classification

1. **FlowLayout – arranges components in a row**
2. **BorderLayout – divides area into North, South, East, West, Center**
3. **GridLayout – arranges components in rows and columns**

Explanation / Working Principle

Layout managers automatically arrange GUI components based on rules.

Advantages

- **No manual positioning**
- **Responsive design**

Disadvantages

- **Less precise control**

Applications

- **GUI applications**

Conclusion / Summary

Layout managers simplify GUI design in Java.

---

54. Life Cycle of an Applet

Definition / Introduction

Applet is a small Java program that runs in a web browser.

Key Concepts

- **Initialization**
- **Execution phases**

Life Cycle Methods

1. **init()**
2. **start()**
3. **paint()**
4. **stop()**
5. **destroy()**

Explanation / Working Principle

Applet moves through different stages during execution in browser.

Advantages

- **Easy web integration (old concept)**

Disadvantages

- **Now outdated and not widely used**

Applications

- **Old web applications**

Conclusion / Summary

Applet follows a fixed life cycle from init to destroy.

---

55. Difference Between Applet and Application

Definition / Introduction

Applet runs in browser, application runs independently.

Differences

Feature	Applet	Application
Execution	Browser	JVM
Security	Restricted	Full access
Usage	Web	Standalone

Advantages

- **Applications are more powerful**

Disadvantages

- **Applets are outdated**

Applications

- **Desktop software**

Conclusion / Summary

Applications are widely used, applets are obsolete.

---

56. Event Source and Event Listener

Definition / Introduction

Event source generates events, event listener handles them.

Key Concepts

- **Event source: button, keyboard**
- **Event listener: handles event**

Explanation / Working Principle

User action generates event which is captured by listener.

Syntax

```
button.addActionListener(listener);
```

Advantages

- **Interactive applications**

Disadvantages

- **Complex event management**

Applications

- **GUI systems**

Conclusion / Summary

Event source generates events and listener processes them.

---

57. Adapter Classes in Java

Definition / Introduction

Adapter classes are special classes in Java that provide empty implementations of listener interfaces.

Key Concepts

- **Used in event handling**
- **Reduce code complexity**

Explanation / Working Principle

Instead of implementing all methods of an interface, adapter classes allow overriding only required methods.

Types / Classification

- **MouseAdapter**
- **KeyAdapter**
- **WindowAdapter**

Advantages

- **Reduces code length**
- **Easy to use**

Disadvantages

- **Still requires inheritance**

Applications

- **GUI event handling**

Conclusion / Summary

Adapter classes simplify event handling by providing default method implementations.

---

58. JPanel, JFrame, JApplet in Java

Definition / Introduction

These are Swing components used to build GUI applications.

Key Concepts

- **JFrame: main window**
- **JPanel: container inside frame**
- **JApplet: applet-based GUI container (obsolete)**

Explanation / Working Principle

JFrame is the top-level window, JPanel is used to organize components inside it.

Differences

Component	Use
JFrame	Main window
JPanel	Sub-container
JApplet	Web-based GUI (old)

Advantages

- **Modular GUI design**

Disadvantages

- **Applet is outdated**

Applications

- **Desktop GUI applications**

Conclusion / Summary

JFrame and JPanel are essential Swing components for GUI design.

---

59. Passing Parameters to Applets

Definition / Introduction

Applet parameters allow passing values from HTML to Java applet.

Key Concepts

- **HTML tag**
- **getParameter() method**

Explanation / Working Principle

Values are defined in HTML and read by applet using getParameter().

Syntax

HTML:

```
<param name="name" value="Java">
```

Java:

```
String val = getParameter("name");
```

Advantages

- **Dynamic input to applet**

Disadvantages

- **Limited and outdated feature**

Applications

- **Old web-based Java programs**

Conclusion / Summary

Applet parameters allow communication between HTML and Java applet.